Sum Bsts

AlgoExpert · Hard · Binary Search Trees

The problem

- Sum the values of every node that roots a valid BST subtree containing at least 3 nodes.
- Post-order collecting (isBst, size, min, max, sum).

Intuition

- Post-order lets each node know whether both children are BSTs, their sizes, sums and value ranges.

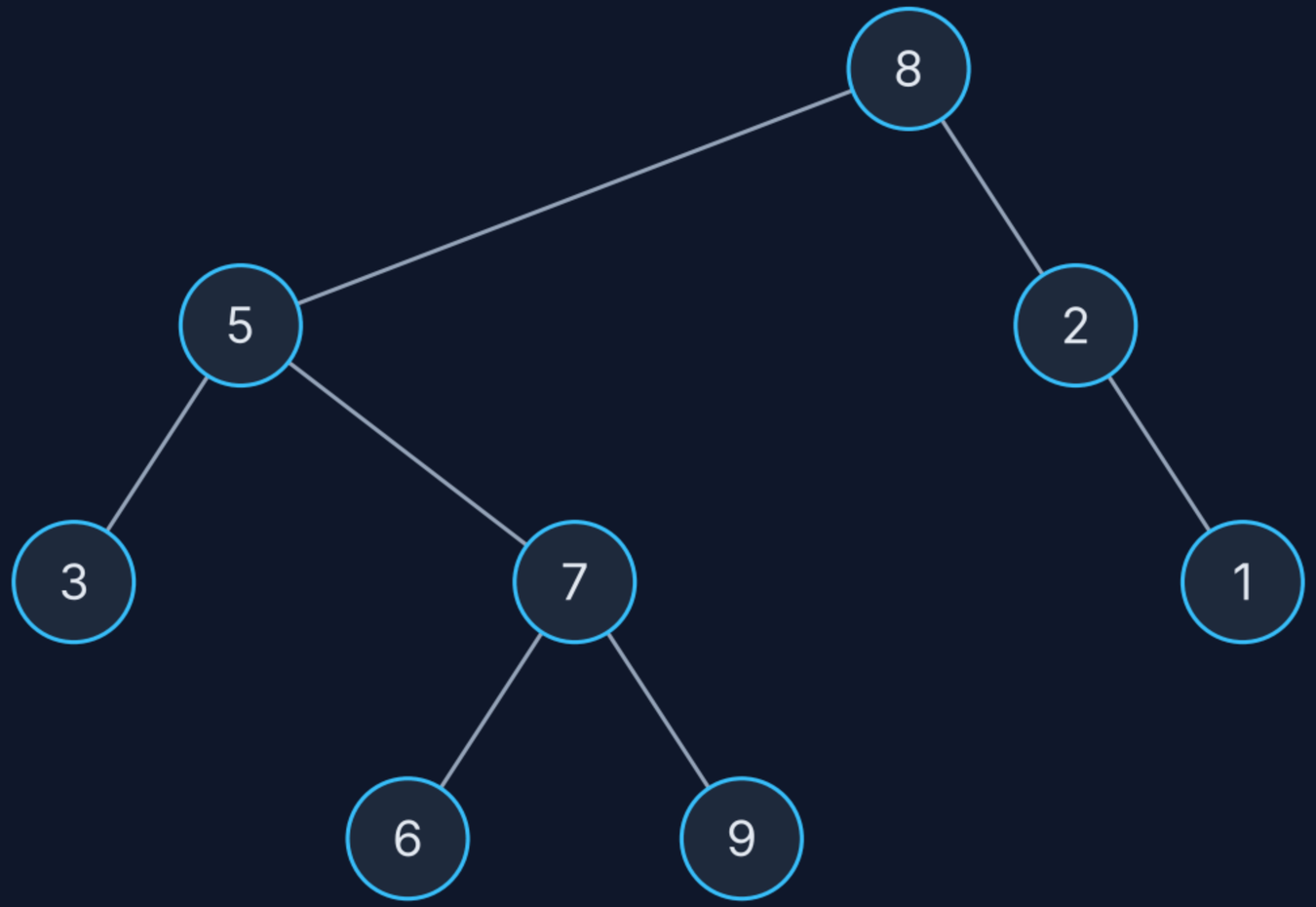
Brute force vs optimal

- For each node validate subtree, then sum → $O(n^2)$
- Single post-order pass returning (isBst, size, min, max, sum) → $O(n)$

Approach

- 1. Return validity, size, sum, min and max from each subtree
- 2. Combine at the parent
- 3. Add sums of valid BSTs with three or more nodes

Worked example

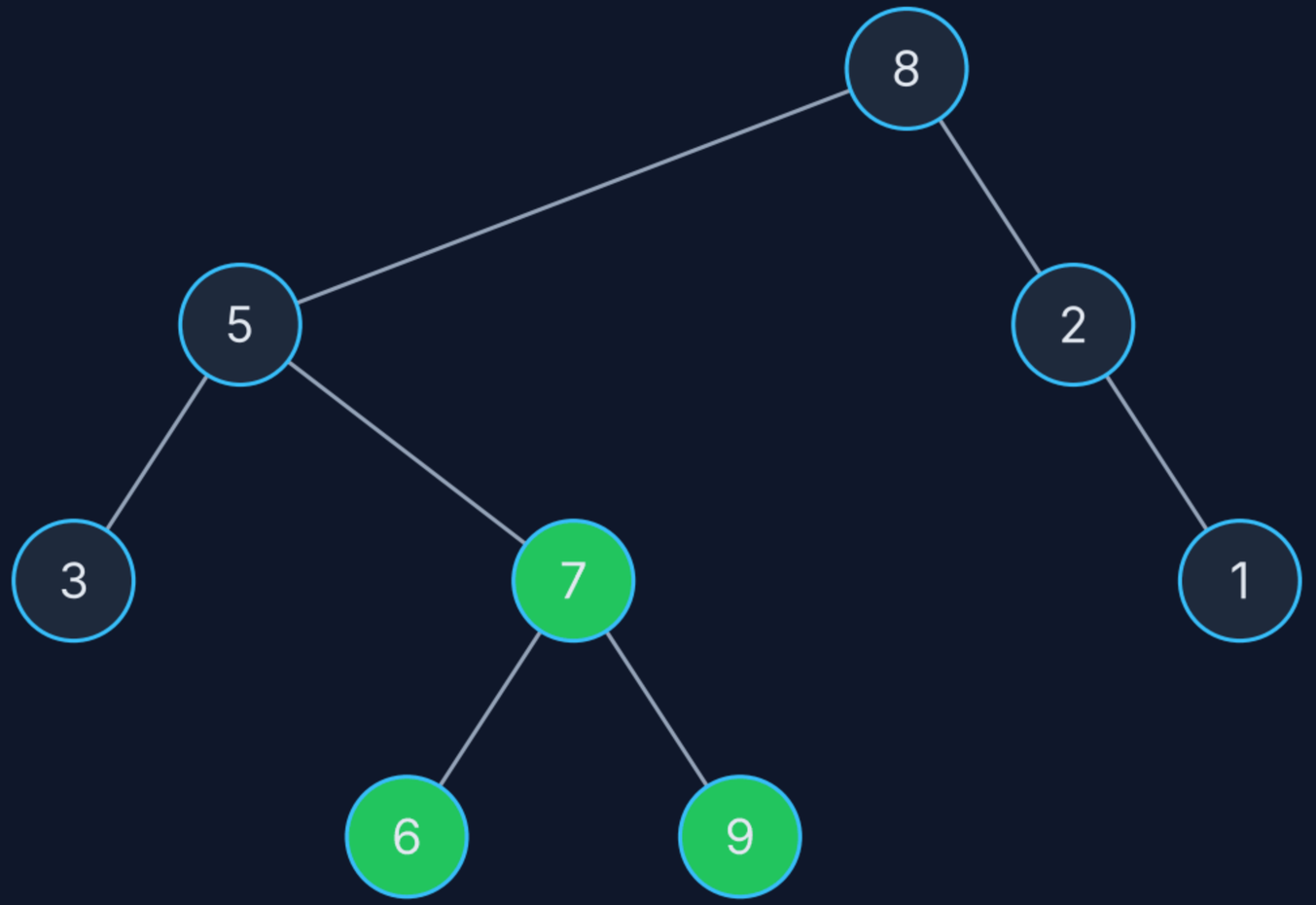


post-order: each node returns

(isBst, size, min, max, sum)

Post-order: return BST info upward

Worked example

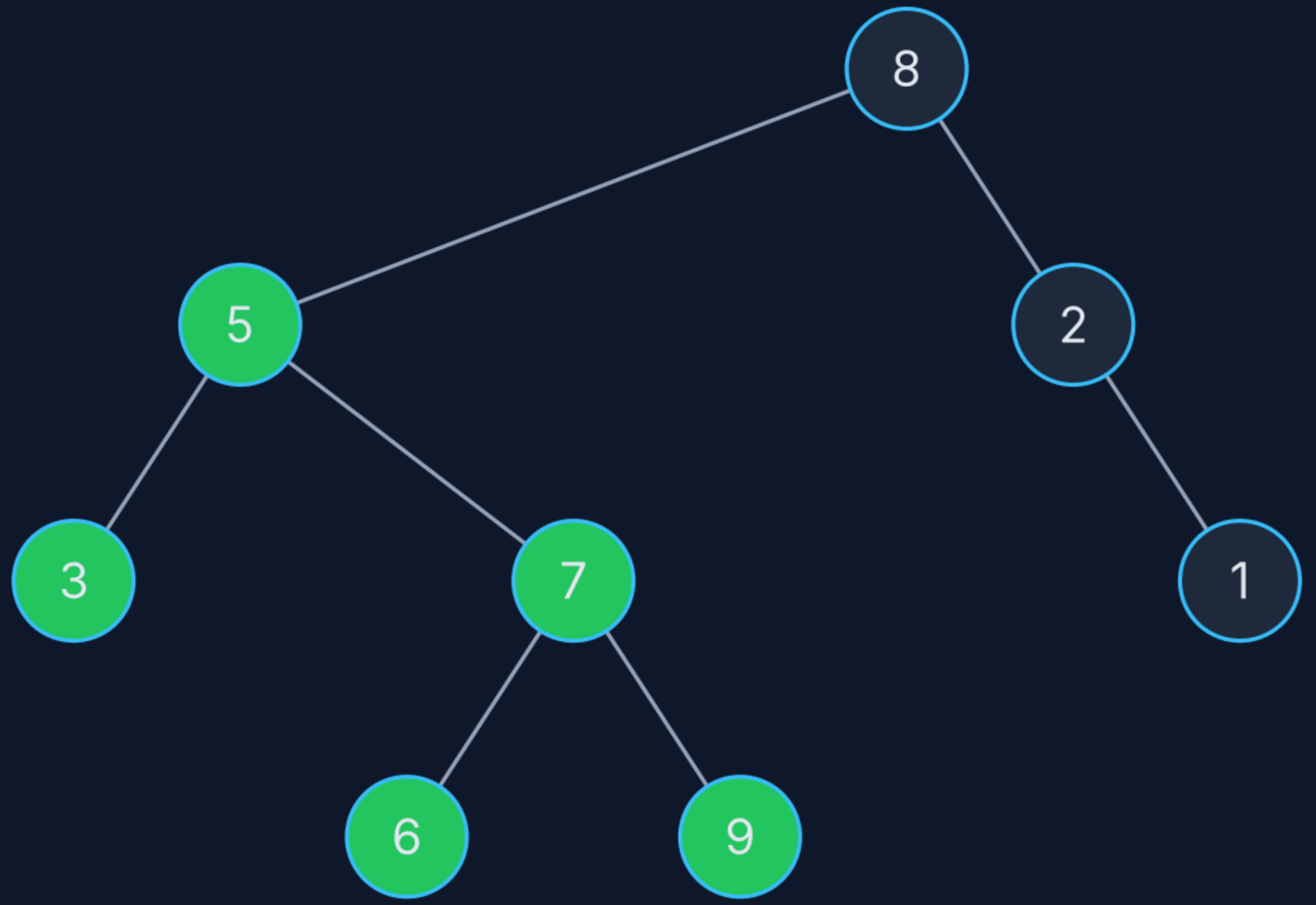



```
subtree 7: {6, 7, 9}

BST, size 3 → sum 22
```

Subtree at 7 is a BST of size 3 → 22

Worked example



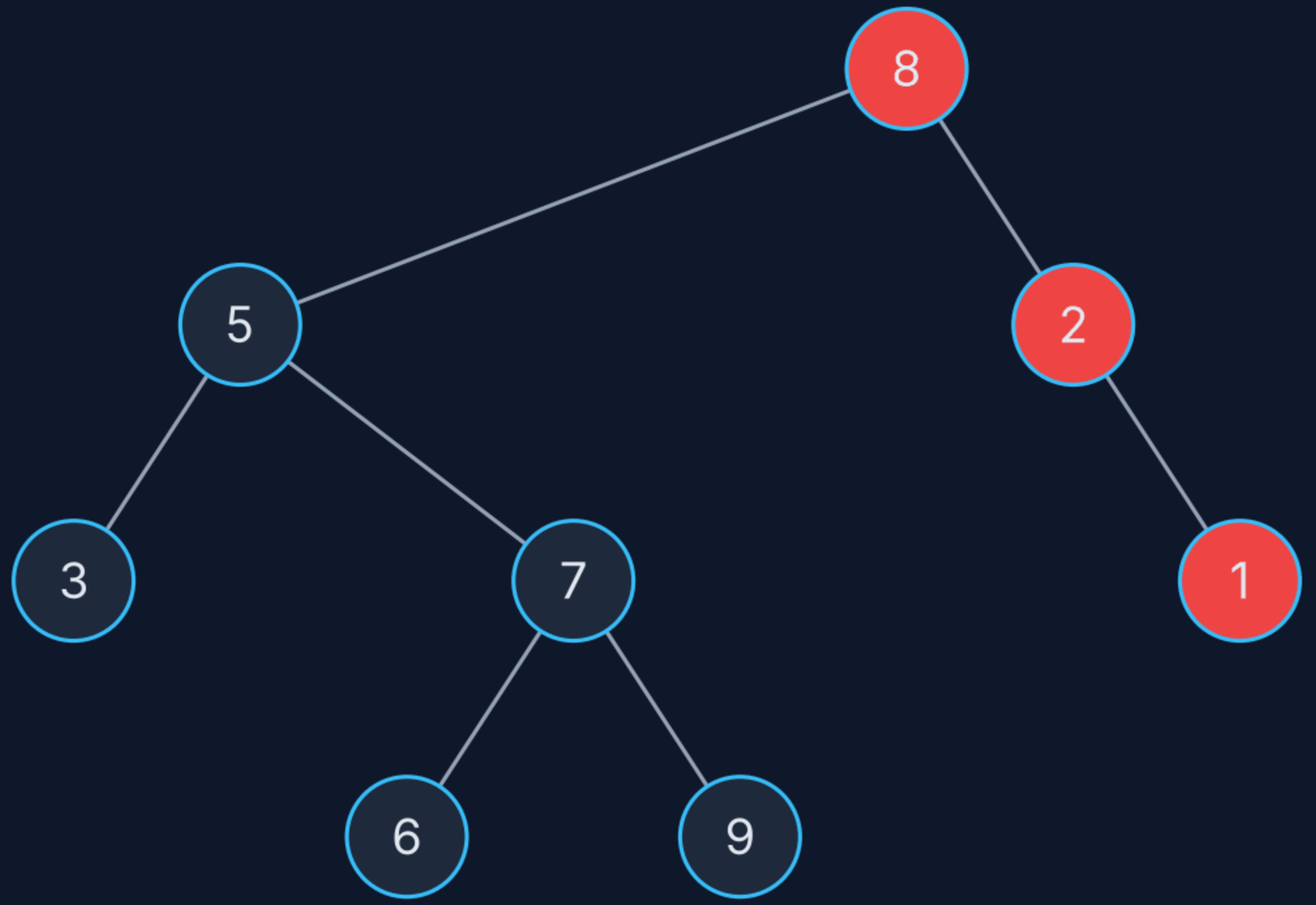

```
subtree 5: 3 < 5 ≤ 6 ✓

BST, size 5 → sum 30

(replaces the 22 inside it)
```

Subtree at 5 is a bigger BST → 30

Worked example




```
subtree 2: right child 1 < 2 ✗

root 8: right side invalid ✗

answer = 30
```

Other subtrees fail → answer 30

Java solution

```java
public class SumBsts {
    public static int sumBsts(BinaryTree t) { long[] total = {0}; info(t, total); return (int) total[0]; }

    /** returns {isBst, size, min, max, sum} */
    private static long[] info(BinaryTree t, long[] total) {
        if (t == null) return new long[]{1, 0, Long.MAX_VALUE, Long.MIN_VALUE, 0};
        long[] l = info(t.left, total), r = info(t.right, total);
        boolean bst = l[0] == 1 && r[0] == 1 && l[3] < t.value && t.value <= r[2];
        long size = l[1] + r[1] + 1, sum = l[4] + r[4] + t.value;
        if (bst && size >= 3) {
            total[0] += sum;
            // children already counted? subtract their counted contributions to avoid double counting
            if (l[1] >= 3) total[0] -= l[4];
            if (r[1] >= 3) total[0] -= r[4];
        }
        return new long[]{bst ? 1 : 0, size, Math.min(l[2], t.value), Math.max(r[3], t.value), sum};
    }
}
```

Complexity

- Time: O(n)
- Space: O(h)

Edge cases & pitfalls

- Only BSTs with ≥ 3 nodes count
- Don't double count nested BSTs
- Empty subtree = valid, size 0

Interview tips & follow-ups

- State the returned tuple clearly

Recap

- Post-order lets each node know whether both children are BSTs, their sizes, sums and value ranges.
- Complexity: O(n)